

Руководство по SSH для проекта MAPS

Сервер: Oracle Cloud — 132.145.232.46

Пользователь: ubuntu

Проект: ~/maps

Шаг 1 — ~/.ssh/config: базовое подключение

Файл ~/.ssh/config позволяет задавать параметры подключения для каждого хоста, чтобы не писать длинные команды каждый раз.

Структура файла

```
Host <псевдоним>
    Параметр1 значение
    Параметр2 значение
```

Отступ (пробелы или таб) обязателен для параметров внутри блока.

Основные параметры

Параметр	Что делает	Пример
HostName	Реальный IP или домен	132.145.232.46
User	Имя пользователя	ubuntu
IdentityFile	Путь к приватному ключу	~/.ssh/id_rsa
Port	Порт (по умолчанию 22)	2222
ForwardAgent	Проброс SSH-агента	yes
ServerAliveInterval	Пинг каждые N секунд	60

Конфиг для MAPS

```
Host maps-server
    HostName 132.145.232.46
    User ubuntu
    IdentityFile ~/keys/oracle.pem
    ServerAliveInterval 60
```

Теперь вместо `ssh ubuntu@132.145.232.46 -i ~/keys/oracle.pem` пишем просто:

```
ssh maps-server
```

Несколько псевдонимов для одного хоста

```
Host oracle maps-server prod
    HostName 132.145.232.46
    User ubuntu
    IdentityFile ~/keys/oracle.pem
```

Все три имени ведут на один сервер.

Wildcard * — общие настройки для всех хостов

```
Host *
    ServerAliveInterval 60
    ServerAliveCountMax 3
    AddKeysToAgent yes
```

Блок `Host *` ставят в конец файла — параметры применяются ко всем хостам, но не перезаписывают уже заданные выше.

Проверка конфига

```
ssh -G maps-server      # показывает все параметры для хоста
ssh -v maps-server      # verbose — подробный лог подключения
```

Шаг 2 — Проброс портов (LocalForward)

На сервере PostgreSQL слушает порт 5432, но он закрыт снаружи. Проброс портов создаёт зашифрованный туннель через SSH.

```
Ваш компьютер      SSH туннель      Сервер Oracle
localhost:5432  <----->  127.0.0.1:5432
```

Способ 1 — Разовая команда

```
ssh -L 5432:localhost:5432 maps-server
```

Пока терминал открыт — туннель работает.

Способ 2 — Туннель в фоне

```
ssh -L 5432:localhost:5432 -N -f maps-server
```

- `-N` — не выполнять команды, только держать туннель
- `-f` — уйти в фон

Закреть фоновый туннель:

```
ps aux | grep "ssh -L"
kill XXXX
```

Способ 3 — Прописать в конфиг (рекомендуется)

```
Host maps-server
    HostName 132.145.232.46
    User ubuntu
    IdentityFile ~/keys/oracle.pem
    ServerAliveInterval 60
    LocalForward 5432 localhost:5432
```

Теперь туннель поднимается автоматически при каждом `ssh maps-server`.

Несколько портов сразу

```
Host maps-server
  HostName 132.145.232.46
  User ubuntu
  IdentityFile ~/.keys/oracle.pem
  ServerAliveInterval 60
  LocalForward 5432 localhost:5432 # PostgreSQL
  LocalForward 8000 localhost:8000 # FastAPI
```

Подключение к PostgreSQL через туннель

```
psql -h localhost -p 5432 -U maps_user -d maps_db
```

В DBeaver / pgAdmin:

```
Host:      localhost
Port:      5432
Database:  maps_db
User:      maps_user
```

Типичные ошибки

bind: Address already in use — порт уже занят. Используйте другой локальный порт:

```
ssh -L 5433:localhost:5432 maps-server
```

Проверка что туннель работает

```
ss -tlnp | grep 5432
```

Шаг 3 — Копирование файлов (scp / rsync)

	scp	rsync
Простое копирование одного файла	удобно	избыточно
Папка с файлами	неудобно	удобно
Только изменения (дельта)	нет	да
Прогресс-бар	нет	да
Исключить файлы	нет	да

scp — простое копирование

```
# Загрузить .env на сервер
scp .env maps-server:~/maps/.env
# Загрузить Excel-файл
scp materials.xlsx maps-server:~/maps/data/
# Скачать лог
```

```
scp maps-server:~/maps/logs/app.log ./app.log
# Скачать бэкап БД
scp maps-server:~/backups/maps_backup.sql ./
# Скопировать папку целиком
scp -r maps-server:~/maps/logs ./logs
```

rsync — умная синхронизация

Основные флаги:

Флаг	Что делает
-a	архивный режим — сохраняет права, время, рекурсия
-v	verbose — показывает что копируется
-z	сжатие при передаче
-P	прогресс-бар + продолжение при обрыве
--delete	удалить на сервере то чего нет локально
--exclude	исключить файлы/папки
--dry-run	показать что будет сделано, ничего не меняя

На практике почти всегда используют -avzP.

Синхронизировать проект MAPS на сервер

```
rsync -avzP \
  --exclude='.env' \
  --exclude='__pycache__/' \
  --exclude='*.pyc' \
  --exclude='.git/' \
  --exclude='venv/' \
  --exclude='*.egg-info/' \
  ./ maps-server:~/maps/
```

.env исключён намеренно — на сервере хранится ANTHROPIC_API_KEY.

Проверить что будет скопировано (без реального копирования)

```
rsync -avzP --dry-run --exclude='.env' ./ maps-server:~/maps/
```

Скачать бэкап БД

```
ssh maps-server "pg_dump -U maps_user maps_db > ~/backups/maps_$(date +%Y%m%d).sql"
rsync -avzP maps-server:~/backups/ ./backups/
```

Деплой + перезапуск одной командой

```
rsync -avzP \
  --exclude='.env' --exclude='__pycache__/' \
  --exclude='*.pyc' --exclude='.git/' \
  --exclude='venv/' --exclude='*.egg-info/' \
```

```
./ maps-server:~/maps/ && \  
ssh maps-server "cd ~/maps && source venv/bin/activate && \  
pip install -e . -q && systemctl restart maps && systemctl status maps"
```

Шаг 4 — Выполнение команд без входа на сервер

```
ssh maps-server "команда"
```

Команда выполняется на сервере, результат выводится локально.

Управление сервисом MAPS

```
# Статус  
ssh maps-server "systemctl status maps"  
# Перезапустить  
ssh maps-server "sudo systemctl restart maps"  
# Остановить / запустить  
ssh maps-server "sudo systemctl stop maps"  
ssh maps-server "sudo systemctl start maps"
```

Логи

```
# Последние 50 строк  
ssh maps-server "journalctl -u maps -n 50"  
# В реальном времени (Ctrl+C для выхода)  
ssh maps-server "journalctl -u maps -f"
```

Состояние сервера

```
# Использование диска  
ssh maps-server "df -h"  
# Использование памяти  
ssh maps-server "free -h"  
# Нагрузка на CPU  
ssh maps-server "top -bn1 | head -20"  
# Слушает ли порт 8000  
ssh maps-server "ss -tlnp | grep 8000"
```

Несколько команд

```
# && — остановиться при ошибке (рекомендуется)  
ssh maps-server "systemctl stop maps && systemctl start maps"  
# ; — выполнить все, даже при ошибке  
ssh maps-server "systemctl stop maps ; systemctl start maps"
```

Многострочные команды (heredoc)

```
ssh maps-server << 'EOF'  
cd ~/maps  
source venv/bin/activate  
pip install -e . -q  
systemctl restart maps  
systemctl status maps
```

EOF

Одинарные кавычки вокруг EOF важны — без них переменные \$VAR раскроются локально.

Запустить Cloudflare туннель

```
ssh maps-server "cd ~/maps && bash deploy/tunnel.sh --bg && cat /tmp/maps_tunnel.url"
```

Получить вывод в локальную переменную

```
STATUS=$(ssh maps-server "systemctl is-active maps")
echo "Статус MAPS: $STATUS"
TUNNEL_URL=$(ssh maps-server "cat /tmp/maps_tunnel.url 2>/dev/null")
echo "MAPS доступен по: $TUNNEL_URL"
```

Типичная ошибка — переменные

```
# Неправильно — $HOME раскроется локально
ssh maps-server "ls $HOME/maps"
# Правильно — одинарные кавычки
ssh maps-server 'ls $HOME/maps'
```

Полный деплой одной командой

```
ssh maps-server << 'EOF'
cd ~/maps
git pull origin main
source venv/bin/activate
pip install -e . -q
sudo systemctl restart maps
echo "=== Статус ==="
systemctl status maps --no-pager
EOF
```

Шаг 5 — SSH-агент и ForwardAgent

Что такое SSH-агент

SSH-агент — фоновый процесс, хранящий расшифрованные ключи в памяти.

Без агента: `ssh maps-server` → "введите passphrase" → подключение
С агентом: `ssh maps-server` → подключение (ключ уже в памяти)

Запустить агент и добавить ключ

```
eval "$(ssh-agent -s)"
ssh-add ~/keys/oracle.pem
# Проверить загруженные ключи
ssh-add -l
# Удалить все ключи
ssh-add -D
```

Автозапуск агента (добавить в ~/.bashrc)

```
if [ -z "$SSH_AUTH_SOCK" ]; then
    eval "$(ssh-agent -s)" > /dev/null
    ssh-add ~/keys/oracle.pem 2>/dev/null
fi
```

AddKeysToAgent в конфиге

```
Host maps-server
    HostName 132.145.232.46
    User ubuntu
    IdentityFile ~/keys/oracle.pem
    AddKeysToAgent yes
    ServerAliveInterval 60
```

При первом ssh maps-server ключ автоматически добавится в агент.

ForwardAgent — проброс агента на сервер

Позволяет делать git pull на сервере используя ваш локальный GitHub-ключ. Ключи не копируются на сервер — сервер обращается к агенту через туннель.

```
Ваш компьютер                               Сервер Oracle
[SSH-агент с ключами] <-----> git pull origin main
```

Включить ForwardAgent

```
Host maps-server
    HostName 132.145.232.46
    User ubuntu
    IdentityFile ~/keys/oracle.pem
    AddKeysToAgent yes
    ForwardAgent yes
    ServerAliveInterval 60
```

Проверить что ForwardAgent работает

```
# На сервере:
ssh -T git@github.com
# Hi DaniarSher81212! You've successfully authenticated...
```

Безопасность

- Включайте ForwardAgent только на доверенных серверах
- Не используйте ForwardAgent yes в блоке Host *
- При компрометации сервера: ssh-add -D (удалить все ключи из агента)

Шаг 6 — Алиасы и скрипты

Алиасы в ~/.bashrc

```
# ===== MAPS алиасы =====
alias maps-ssh='ssh maps-server'
alias maps-status='ssh maps-server "systemctl status maps --no-pager"'
alias maps-restart='ssh maps-server "sudo systemctl restart maps"'
alias maps-stop='ssh maps-server "sudo systemctl stop maps"'
alias maps-start='ssh maps-server "sudo systemctl start maps"'
alias maps-logs='ssh maps-server "journalctl -u maps -n 100 --no-pager"'
alias maps-logs-live='ssh maps-server "journalctl -u maps -f"'
alias maps-stats='ssh maps-server "echo === ДИСК === && df -h && echo === ПАМЯТЬ === && free -h && echo === НАПРЯЖЕННОСТЬ === && iostat -x 1 1 && echo ==="'
alias maps-url='ssh maps-server "cat /tmp/maps_tunnel.url 2>/dev/null || echo туннель не запущен"'
alias maps-tunnel='ssh maps-server "cd ~/maps && bash deploy/tunnel.sh --bg && sleep 2 && cat /tmp/maps_tunnel.url"'
```

Применить: `source ~/.bashrc`

Папка для скриптов

```
mkdir -p ~/scripts
```

Добавить в `~/.bashrc`:

```
export PATH="$HOME/scripts:$PATH"
```

Скрипт деплоя — maps-deploy

Файл `~/scripts/maps-deploy`:

```
#!/bin/bash
set -e
echo "==> Синхронизация кода..."
rsync -avzP \
  --exclude='.env' \
  --exclude='__pycache__/' \
  --exclude='*.pyc' \
  --exclude='.git/' \
  --exclude='venv/' \
  --exclude='*.egg-info/' \
  ./ maps-server:~/maps/
echo "==> Установка зависимостей и перезапуск..."
ssh maps-server << 'EOF'
cd ~/maps
source venv/bin/activate
pip install -e . -q
sudo systemctl restart maps
sleep 2
systemctl status maps --no-pager
EOF
echo "==> Готово!"
```

```
chmod +x ~/scripts/maps-deploy
```

Скрипт бэкапа — maps-backup

Файл `~/scripts/maps-backup`:

```
#!/bin/bash
BACKUP_DIR="$HOME/backups/maps"
DATE=$(date +%Y%m%d_%H%M)
FILENAME="maps_${DATE}.sql"
```



```

mkdir -p "$BACKUP_DIR"
echo "==> Создание дампа на сервере..."
ssh maps-server "pg_dump -U maps_user maps_db > ~/backups/${FILENAME}"
echo "==> Скачивание дампа..."
rsync -avzP maps-server:~/backups/${FILENAME} "${BACKUP_DIR}/${FILENAME}"
echo "==> Оставить только последние 10 бэкапов..."
ls -t "${BACKUP_DIR}"/*.sql | tail -n +11 | xargs -r rm
echo "==> Готово: ${BACKUP_DIR}/${FILENAME}"

```

Скрипт логов с аргументами — maps-logs

Файл ~/scripts/maps-logs:

```

#!/bin/bash
LINES=${1:-100}
if [ "$1" == "-f" ] || [ "$1" == "--follow" ]; then
    ssh maps-server "journalctl -u maps -f"
else
    ssh maps-server "journalctl -u maps -n ${LINES} --no-pager"
fi

```

Использование:

```

maps-logs          # последние 100 строк
maps-logs 200      # последние 200 строк
maps-logs -f        # в реальном времени

```

Скрипт диагностики — maps-check

Файл ~/scripts/maps-check:

```

#!/bin/bash
echo "=====
echo "  MAPS — диагностика $(date '+%Y-%m-%d %H:%M') "
echo "=====
ssh maps-server << 'EOF'
echo ""
echo "--- Сервис MAPS ---"
systemctl status maps --no-pager | head -20
echo ""
echo "--- Порт 8000 ---"
ss -tlnp | grep 8000 || echo "порт не слушается!"
echo ""
echo "--- PostgreSQL ---"
systemctl status postgresql --no-pager | head -5
echo ""
echo "--- Диск ---"
df -h | grep -E "Filesystem|/$"
echo ""
echo "--- Память ---"
free -h
echo ""
echo "--- Туннель ---"
cat /tmp/maps_tunnel.url 2>/dev/null || echo "туннель не запущен"
echo ""
echo "--- Последние ошибки ---"
journalctl -u maps -n 20 --no-pager | grep -i "error\|exception\|critical" || echo "ошибок нет"
EOF

```

Итог — все команды

maps-ssh	# войти на сервер
maps-status	# статус сервиса
maps-restart	# перезапустить
maps-stop	# остановить
maps-start	# запустить
maps-logs	# последние 100 строк логов
maps-logs 200	# последние 200 строк
maps-logs -f	# логи в реальном времени
maps-stats	# диск, память, нагрузка
maps-url	# текущий URL туннеля
maps-tunnel	# запустить туннель
maps-deploy	# задеплоить изменения
maps-backup	# бэкап базы данных
maps-check	# полная диагностика

Шаг 7 — Безопасность SSH

Проверить текущие настройки сервера

```
ssh maps-server "sudo grep -E 'PasswordAuth|PermitRoot|Port' /etc/ssh/sshd_config"
```

Хорошие значения:

```
PasswordAuthentication no
PermitRootLogin no
Port 22
```

Основные настройки /etc/ssh/sshd_config

```
# Только вход по ключу
PasswordAuthentication no
# Запрет входа под root
PermitRootLogin no
# Разрешить только конкретного пользователя
AllowUsers ubuntu
# Максимум попыток аутентификации
MaxAuthTries 3
# Таймаут неактивного подключения
ClientAliveInterval 300
ClientAliveCountMax 2
# Отключить пустые пароли
PermitEmptyPasswords no
# Отключить X11 forwarding
X11Forwarding no
```

Применить: `sudo systemctl restart sshd`

Важно: перед закрытием сессии проверьте новое подключение в другом терминале:

```
ssh maps-server "echo OK"
```

Сменить порт SSH

В /etc/ssh/sshd_config:

Port 2222

Обязательно:

1. Открыть порт 2222 в Security List Oracle Cloud (веб-консоль)
2. Открыть порт в UFW: `sudo ufw allow 2222/tcp`
3. Обновить ~/.ssh/config локально: добавить Port 2222
4. Проверить: `ssh -p 2222 maps-server "echo OK"` — только потом закрывать порт 22

UFW файрвол

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 2222/tcp      # SSH
sudo ufw allow 80/tcp       # HTTP
sudo ufw allow 443/tcp      # HTTPS (Cloudflare Tunnel)
sudo ufw enable
sudo ufw status verbose
```

Порты 8000 (FastAPI) и 5432 (PostgreSQL) не открываем — доступны только через туннели.

Fail2ban — защита от брутфорса

Установка:

```
sudo apt install fail2ban -y
```

Конфиг /etc/fail2ban/jail.local:

```
[DEFAULT]
bantime  = 3600
findtime = 600
maxretry = 5
[sshd]
enabled  = true
port     = 2222
logpath  = /var/log/auth.log
maxretry = 3
```

```
sudo systemctl enable fail2ban
sudo systemctl start fail2ban
```

Проверить заблокированные IP:

```
sudo fail2ban-client status sshd
```

Разблокировать IP:

```
sudo fail2ban-client set sshd unbanip 1.2.3.4
```

Мониторинг попыток взлома

```
# Неудачные попытки входа
ssh maps-server "sudo journalctl -u sshd | grep 'Failed password' | tail -20"
# Успешные входы
```

```
ssh maps-server "sudo journalctl -u sshd | grep 'Accepted' | tail -20"
```

```
# Top атакующих IP
```

```
ssh maps-server "sudo journalctl -u sshd | grep 'Failed' | awk '{print \$11}' | sort | uniq -c | sort -rn | head -10"
```

Чек-лист безопасности

- Вход только по SSH-ключу (PasswordAuthentication no)
 - Вход под root запрещён (PermitRootLogin no)
 - Разрешён только пользователь ubuntu (AllowUsers ubuntu)
 - SSH на нестандартном порту (Port 2222)
 - UFW фаервол включён
 - Порты 8000 и 5432 закрыты снаружи
 - Fail2ban защищает от брутфорса
 - Cloudflare Tunnel — единственный внешний доступ к MAPS
-

Итоговый ~/.ssh/config

```
Host maps-server
    HostName 132.145.232.46
    User ubuntu
    Port 2222
    IdentityFile ~/.keys/oracle.pem
    AddKeysToAgent yes
    ForwardAgent yes
    ServerAliveInterval 60
    LocalForward 5432 localhost:5432
Host *
    ServerAliveInterval 60
    ServerAliveCountMax 3
    AddKeysToAgent yes
```